

Project: AI-Powered Bone Fracture Detection

Authors: Tanishq Ray & Maithli Singh Tomar

Objective: To develop a robust, clinically interpretable Deep Learning model (*DenseNet121*) for automating fracture diagnosis from X-ray imagery, utilizing Class Weights for imbalance handling and *Grad-CAM* for visual explainability.

Step 1: Environment Setup & High-Speed Data Loading

This step mounts Google Drive and copies the dataset to the local Colab runtime to prevent I/O bottlenecks during training.

```
from google.colab import drive, files
from IPython.display import display

import os
import zipfile

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

import cv2
from PIL import ImageFile

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import DenseNet121
from tensorflow.keras import layers, models, optimizers
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping,
ReduceLROnPlateau, CSVLogger
from tensorflow.keras.models import load_model

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.multioutput import MultiOutputClassifier
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_curve,
    auc,
    precision_recall_curve,
    precision_recall_fscore_support
)
```

```

# --- 1.1 Mount Google Drive ---

drive.mount('/content/drive')

# --- 1.2 Configuration ---
# Update these paths to match your exact Drive structure
DRIVE_ARCHIVE_PATH = '/content/drive/MyDrive/X-Ray ML
Project/FracAtlas/images.zip'
DRIVE_CSV_PATH = '/content/drive/MyDrive/X-Ray ML
Project/FracAtlas/dataset.csv'
LOCAL_DATA_DIR = '/content/dataset'

# --- 1.3 High-Speed Local Extraction ---
# Unzipping to local disk (/content) is 100x faster than reading from
Drive
if not os.path.exists(LOCAL_DATA_DIR):
    print(f" Copying dataset from {DRIVE_ARCHIVE_PATH} to local
disk...")
    !cp "{DRIVE_ARCHIVE_PATH}" "/content/images.zip"

    print(" Unzipping...")
    with zipfile.ZipFile("/content/images.zip", 'r') as zip_ref:
        zip_ref.extractall(LOCAL_DATA_DIR)
    print(" Data ready at:", LOCAL_DATA_DIR)
else:
    print(" Data already extracted.")

Mounted at /content/drive
Copying dataset from /content/drive/MyDrive/X-Ray ML
Project/FracAtlas/images.zip to local disk...
Unzipping...
Data ready at: /content/dataset

```

Step 2: Data Handling & Stratified Splitting

We load the metadata and perform a Stratified Shuffle Split to ensure the 1:5 fracture-to-healthy ratio is consistent across Training, Validation, and Test sets.

```

# Allow loading of slightly corrupted/truncated images
ImageFile.LOAD_TRUNCATED_IMAGES = True

# --- 2.1 Load Metadata ---
df = pd.read_csv(DRIVE_CSV_PATH)

# --- 2.2 Path Mapping ---
# Map image IDs to their local file paths
def get_local_path(row):

```

```

    # Adjust folder names 'Fractured'/'Non_fractured' based on your
zip structure
    category_folder = 'Fractured' if row['fractured'] == 1 else
'Non_fractured'
    return os.path.join(LOCAL_DATA_DIR, 'images', category_folder,
row['image_id'])

df['image_path'] = df.apply(get_local_path, axis=1)

# --- 2.3 Stratified Split (80% Train, 10% Val, 10% Test) ---
train_df, temp_df = train_test_split(df, test_size=0.2,
stratify=df['fractured'], random_state=42)
val_df, test_df = train_test_split(temp_df, test_size=0.5,
stratify=temp_df['fractured'], random_state=42)

print(f" Dataset Split:\n Train: {len(train_df)}\n Val:
{len(val_df)}\n Test:  {len(test_df)}")

Dataset Split:
Train: 3266
Val: 408
Test: 409

```

Step 2 (B) : Dataset Statistics (EDA)

```

# --- 1. Define the labels first so the function can see them ---
target_columns = [
    'hand', 'leg', 'hip', 'shoulder', 'mixed',
    'hardware', 'multiscan', 'fractured',
    'frontal', 'lateral', 'oblique'
]

def plot_dataset_statistics():
    # 2. Calculate counts using the target_columns defined above
    label_counts =
df[target_columns].sum().sort_values(ascending=False)

    plt.figure(figsize=(12, 6))

    # Using 'hue' to comply with Seaborn v0.13+
    sns.barplot(
        x=label_counts.values,
        y=label_counts.index,
        hue=label_counts.index,
        palette='viridis',
        legend=False
    )

```

```

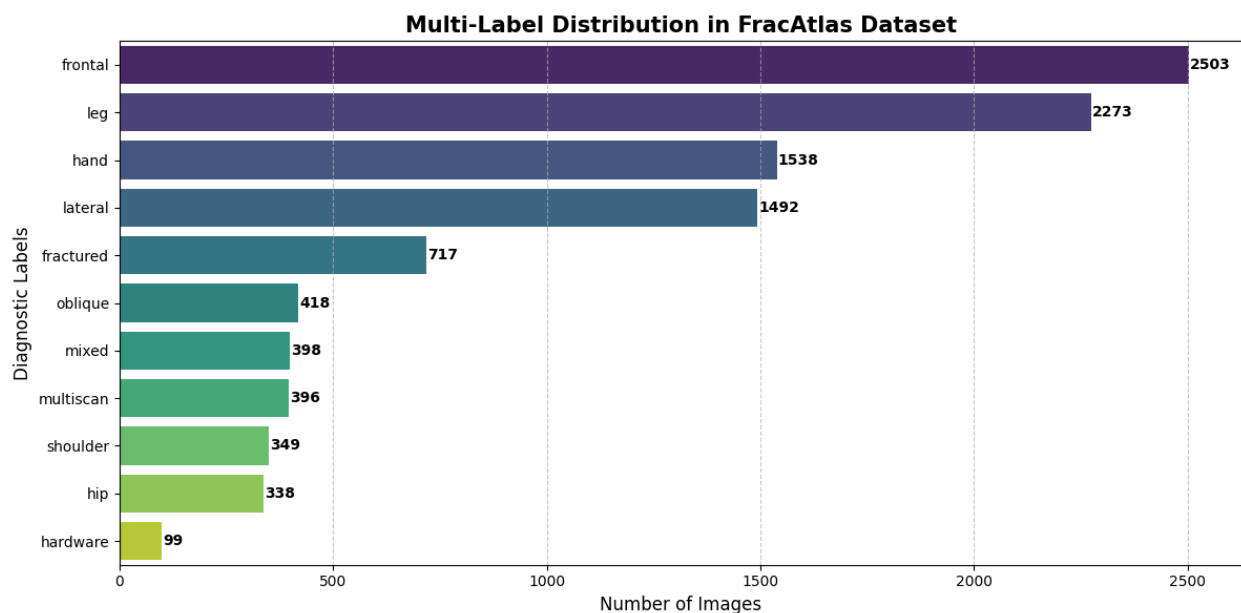
plt.title("Multi-Label Distribution in FracAtlas Dataset",
fontsize=15, fontweight='bold')
plt.xlabel("Number of Images", fontsize=12)
plt.ylabel("Diagnostic Labels", fontsize=12)
plt.grid(axis='x', linestyle='--', alpha=0.7)

# 3. Add data labels for professional reporting
for i, v in enumerate(label_counts.values):
    plt.text(v + 3, i, str(int(v)), color='black', va='center',
fontweight='bold')

plt.tight_layout()
plt.show()

# Run the visualization
plot_dataset_statistics()

```



Step 3: Medical Preprocessing & Generators

Defines the CLAHE (Contrast Limited Adaptive Histogram Equalization) filter and creates data streams. Augmentation is applied only to training data.

```

print(f" Training on {len(target_columns)} labels: {target_columns}")

# --- 3.2 Custom Medical Filter (CLAHE) ---
def medical_clahe_preprocessing(img):
    """
    Applies CLAHE to enhance bone edges and normalizes to [0,1].
    Expected input: RGB image from Keras generator.
    """

```

```

    img = img.astype('uint8')
    # Convert to Grayscale for CLAHE, then stack back to RGB
    gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    enhanced = clahe.apply(gray)
    # Stack 3 channels (required for Transfer Learning backbones)
    return np.stack((enhanced,)*3, axis=-1).astype('float32') / 255.0

# --- 3.3 Data Generators ---
BATCH_SIZE = 32
IMG_SIZE = (224, 224)

# Train Generator: With Augmentation
train_datagen = ImageDataGenerator(
    preprocessing_function=medical_clahe_preprocessing,
    rotation_range=15,
    horizontal_flip=True,
    zoom_range=0.1,
    fill_mode='nearest'
)

# Test/Val Generator: No Augmentation
test_datagen = ImageDataGenerator(
    preprocessing_function=medical_clahe_preprocessing
)

# --- 3.4 Flow from DataFrame (Multi-Label Update) ---
print("< Generating Data Streams...")

# Train Generator
train_gen = train_datagen.flow_from_dataframe(
    train_df,
    x_col='image_path',
    y_col=target_columns, # <--- UPDATED: Pass the list of columns
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='raw', # 'raw' allows multiple output columns
    shuffle=True
)

# Validation Generator
val_gen = test_datagen.flow_from_dataframe(
    val_df,
    x_col='image_path',
    y_col=target_columns, # <--- UPDATED
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='raw',
    shuffle=False
)

```

```
# Test Generator
```

```
test_gen = test_datagen.flow_from_dataframe(  
    test_df,  
    x_col='image_path',  
    y_col=target_columns, # <--- UPDATED  
    target_size=IMG_SIZE,  
    batch_size=BATCH_SIZE,  
    class_mode='raw',  
    shuffle=False  
)
```

```
Training on 11 labels: ['hand', 'leg', 'hip', 'shoulder', 'mixed',  
'hardware', 'multiscan', 'fractured', 'frontal', 'lateral', 'oblique']  
< Generating Data Streams...  
Found 3266 validated image filenames.  
Found 408 validated image filenames.  
Found 409 validated image filenames.
```

```
def visualize_clahe_effect():
```

```
    # 1. Pick a random image from the training set
```

```
    sample_row = train_df.sample(7).iloc[0]
```

```
    img_path = sample_row['image_path']
```

```
    # 2. Load Original Image (Raw)
```

```
    # We load it exactly how the model would see it initially
```

```
    original_img = cv2.imread(img_path)
```

```
    original_img = cv2.cvtColor(original_img, cv2.COLOR_BGR2RGB)
```

```
    # 3. Apply the CLAHE Filter (Same logic as Step 3)
```

```
    # We replicate the function logic here to see the intermediate
```

```
    result
```

```
    gray = cv2.cvtColor(original_img, cv2.COLOR_RGB2GRAY)
```

```
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
```

```
    enhanced_gray = clahe.apply(gray)
```

```
    # 4. Display Side-by-Side Comparison
```

```
    plt.figure(figsize=(12, 6))
```

```
    # Plot Original
```

```
    plt.subplot(1, 2, 1)
```

```
    plt.imshow(original_img)
```

```
    plt.title(f"Original X-Ray\n(Low Contrast)", fontsize=12)
```

```
    plt.axis('off')
```

```
    # Plot CLAHE Enhanced
```

```
    plt.subplot(1, 2, 2)
```

```
    plt.imshow(enhanced_gray, cmap='gray') # Show as grayscale to see
```

```

details
plt.title(f"CLAHE Enhanced\n(Sharper Bone Edges)", fontsize=12)
plt.axis('off')

plt.tight_layout()
plt.show()

print(f"📄 Visualizing File: {img_path}")
print("Pre-processing verified: Local contrast enhanced for bone edge detection.")

# Run the verification
visualize_clahe_effect()

```

Original X-Ray
(Low Contrast)



CLAHE Enhanced
(Sharper Bone Edges)



```

📄 Visualizing File:
/content/dataset/images/Non_fractured/IMG0002682.jpg
Pre-processing verified: Local contrast enhanced for bone edge
detection.

```

Step 4: Model Architecture (Transfer Learning)

Builds a DenseNet121 model with a custom classification head designed to prevent overfitting.

```

def build_multilabel_model(num_classes):
    # Load Pre-trained Backbone (Frozen)

```

```

    base_model = DenseNet121(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))
    base_model.trainable = False

    # Custom Head
    x = layers.GlobalAveragePooling2D()(base_model.output)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.4)(x) # Regularization
    x = layers.Dense(512, activation='relu')(x) # Increased capacity
for 11 tasks
    x = layers.Dropout(0.3)(x)

    # Multi-Label Output (11 Neurons, Sigmoid Activation)
    # Sigmoid allows independent probabilities for each class (e.g.,
Hand=0.9, Fracture=0.8)
    output = layers.Dense(num_classes, activation='sigmoid')(x)

    model = models.Model(inputs=base_model.input, outputs=output)

    model.compile(
        optimizer=optimizers.Adam(learning_rate=0.001),
        loss='binary_crossentropy', # Standard loss for multi-label
tasks
        metrics=['accuracy', tf.keras.metrics.Recall(name='recall'),
tf.keras.metrics.AUC(name='auc')]
    )
    return model

# Initialize model using the column list from Step 3
model = build_multilabel_model(num_classes=len(target_columns))
print(f"Multi-Label Model Built. Output Neurons:
{model.output_shape[-1]}")
print(f"Trainable Params: {model.count_params()}")

Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/densenet/
densenet121_weights_tf_dim_ordering_tf_kernels_notop.h5
29084464/29084464 _____ 0s 0us/step
Multi-Label Model Built. Output Neurons: 11
Trainable Params: 7572043

```

Step 5: Training with Persistence & Class Weights

Calculates class weights to handle imbalance and trains the model with Checkpointing (saving progress) and Early Stopping.

```

# --- 5.1 Callbacks ---
checkpoint_path = '/content/drive/MyDrive/X-Ray ML

```

```

Project/best_multilabel_model.keras'
log_path = '/content/drive/MyDrive/X-Ray ML Project/training_log.csv'

callbacks = [
    ModelCheckpoint(checkpoint_path, monitor='val_loss',
save_best_only=True, mode='min', verbose=1),
    EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=3,
min_lr=1e-6),
    # This saves the history to a file so we can plot it later without
retraining!
    CSVLogger(log_path, append=True)
]

# --- 5.2 Train ---
print(" Starting Multi-Label Training...")
history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=20,
    callbacks=callbacks
)

```

Starting Multi-Label Training...

Epoch 1/20

103/103 ————— 0s 750ms/step - accuracy: 0.4316 - auc: 0.8890 - loss: 0.3463 - recall: 0.7273

Epoch 1: val_loss improved from None to 0.16123, saving model to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 1: finished saving model to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

103/103 ————— 142s 1s/step - accuracy: 0.5233 - auc: 0.9505 - loss: 0.2330 - recall: 0.7845 - val_accuracy: 0.5858 - val_auc: 0.9788 - val_loss: 0.1612 - val_recall: 0.8312 - learning_rate: 0.0010

Epoch 2/20

103/103 ————— 0s 683ms/step - accuracy: 0.5854 - auc: 0.9786 - loss: 0.1464 - recall: 0.8623

Epoch 2: val_loss improved from 0.16123 to 0.11956, saving model to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 2: finished saving model to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

103/103 ————— 75s 726ms/step - accuracy: 0.5845 - auc: 0.9807 - loss: 0.1401 - recall: 0.8701 - val_accuracy: 0.6789 - val_auc: 0.9866 - val_loss: 0.1196 - val_recall: 0.8756 - learning_rate: 0.0010

Epoch 3/20

103/103 _____ 0s 745ms/step - accuracy: 0.6077 - auc: 0.9836 - loss: 0.1288 - recall: 0.8817
Epoch 3: val_loss improved from 0.11956 to 0.09172, saving model to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 3: finished saving model to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

103/103 _____ 81s 787ms/step - accuracy: 0.6127 - auc: 0.9840 - loss: 0.1283 - recall: 0.8832 - val_accuracy: 0.7157 - val_auc: 0.9920 - val_loss: 0.0917 - val_recall: 0.9052 - learning_rate: 0.0010

Epoch 4/20

103/103 _____ 0s 812ms/step - accuracy: 0.6322 - auc: 0.9869 - loss: 0.1175 - recall: 0.8911

Epoch 4: val_loss improved from 0.09172 to 0.08927, saving model to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 4: finished saving model to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

103/103 _____ 88s 855ms/step - accuracy: 0.6314 - auc: 0.9865 - loss: 0.1181 - recall: 0.8916 - val_accuracy: 0.7377 - val_auc: 0.9917 - val_loss: 0.0893 - val_recall: 0.9092 - learning_rate: 0.0010

Epoch 5/20

103/103 _____ 0s 627ms/step - accuracy: 0.6740 - auc: 0.9889 - loss: 0.1072 - recall: 0.9033

Epoch 5: val_loss did not improve from 0.08927

103/103 _____ 68s 654ms/step - accuracy: 0.6675 - auc: 0.9881 - loss: 0.1090 - recall: 0.9015 - val_accuracy: 0.7402 - val_auc: 0.9909 - val_loss: 0.0898 - val_recall: 0.9023 - learning_rate: 0.0010

Epoch 6/20

103/103 _____ 0s 649ms/step - accuracy: 0.6598 - auc: 0.9894 - loss: 0.1015 - recall: 0.9039

Epoch 6: val_loss improved from 0.08927 to 0.07857, saving model to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 6: finished saving model to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

103/103 _____ 71s 689ms/step - accuracy: 0.6503 - auc: 0.9886 - loss: 0.1070 - recall: 0.8996 - val_accuracy: 0.7034 - val_auc: 0.9934 - val_loss: 0.0786 - val_recall: 0.9240 - learning_rate: 0.0010

Epoch 7/20

103/103 _____ 0s 701ms/step - accuracy: 0.6445 - auc: 0.9914 - loss: 0.0961 - recall: 0.9056

Epoch 7: val_loss did not improve from 0.07857

103/103 _____ 75s 727ms/step - accuracy: 0.6571 - auc: 0.9901 - loss: 0.1016 - recall: 0.9002 - val_accuracy: 0.7010 - val_auc: 0.9939 - val_loss: 0.0788 - val_recall: 0.9151 -

```
learning_rate: 0.0010
Epoch 8/20
103/103 _____ 0s 655ms/step - accuracy: 0.6160 - auc:
0.9914 - loss: 0.0918 - recall: 0.9142
Epoch 8: val_loss improved from 0.07857 to 0.07581, saving model to
/content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 8: finished saving model to /content/drive/MyDrive/X-Ray ML
Project/best_multilabel_model.keras
103/103 _____ 72s 697ms/step - accuracy: 0.6286 - auc:
0.9912 - loss: 0.0937 - recall: 0.9125 - val_accuracy: 0.7206 -
val_auc: 0.9942 - val_loss: 0.0758 - val_recall: 0.9141 -
learning_rate: 0.0010
Epoch 9/20
103/103 _____ 0s 644ms/step - accuracy: 0.6462 - auc:
0.9907 - loss: 0.0963 - recall: 0.9104
Epoch 9: val_loss improved from 0.07581 to 0.07267, saving model to
/content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 9: finished saving model to /content/drive/MyDrive/X-Ray ML
Project/best_multilabel_model.keras
103/103 _____ 71s 683ms/step - accuracy: 0.6537 - auc:
0.9911 - loss: 0.0933 - recall: 0.9152 - val_accuracy: 0.7132 -
val_auc: 0.9945 - val_loss: 0.0727 - val_recall: 0.9191 -
learning_rate: 0.0010
Epoch 10/20
103/103 _____ 0s 635ms/step - accuracy: 0.6722 - auc:
0.9914 - loss: 0.0910 - recall: 0.9136
Epoch 10: val_loss did not improve from 0.07267
103/103 _____ 68s 662ms/step - accuracy: 0.6721 - auc:
0.9917 - loss: 0.0922 - recall: 0.9128 - val_accuracy: 0.7500 -
val_auc: 0.9940 - val_loss: 0.0739 - val_recall: 0.9171 -
learning_rate: 0.0010
Epoch 11/20
103/103 _____ 0s 636ms/step - accuracy: 0.6906 - auc:
0.9922 - loss: 0.0885 - recall: 0.9199
Epoch 11: val_loss did not improve from 0.07267
103/103 _____ 76s 737ms/step - accuracy: 0.6953 - auc:
0.9922 - loss: 0.0887 - recall: 0.9185 - val_accuracy: 0.7451 -
val_auc: 0.9939 - val_loss: 0.0747 - val_recall: 0.9171 -
learning_rate: 0.0010
Epoch 12/20
103/103 _____ 0s 634ms/step - accuracy: 0.6844 - auc:
0.9938 - loss: 0.0792 - recall: 0.9284
Epoch 12: val_loss did not improve from 0.07267
103/103 _____ 69s 662ms/step - accuracy: 0.6904 - auc:
0.9924 - loss: 0.0849 - recall: 0.9242 - val_accuracy: 0.7819 -
val_auc: 0.9934 - val_loss: 0.0764 - val_recall: 0.9220 -
learning_rate: 0.0010
```

Epoch 13/20
103/103 _____ 0s 647ms/step - accuracy: 0.7364 - auc:
0.9935 - loss: 0.0783 - recall: 0.9269
Epoch 13: val_loss improved from 0.07267 to 0.07252, saving model
to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 13: finished saving model to /content/drive/MyDrive/X-Ray ML
Project/best_multilabel_model.keras
103/103 _____ 71s 685ms/step - accuracy: 0.7339 - auc:
0.9936 - loss: 0.0788 - recall: 0.9254 - val_accuracy: 0.7623 -
val_auc: 0.9935 - val_loss: 0.0725 - val_recall: 0.9200 -
learning_rate: 2.0000e-04

Epoch 14/20
103/103 _____ 0s 616ms/step - accuracy: 0.7155 - auc:
0.9926 - loss: 0.0839 - recall: 0.9249
Epoch 14: val_loss improved from 0.07252 to 0.07144, saving model
to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 14: finished saving model to /content/drive/MyDrive/X-Ray ML
Project/best_multilabel_model.keras
103/103 _____ 69s 670ms/step - accuracy: 0.7061 - auc:
0.9932 - loss: 0.0815 - recall: 0.9268 - val_accuracy: 0.7451 -
val_auc: 0.9945 - val_loss: 0.0714 - val_recall: 0.9230 -
learning_rate: 2.0000e-04

Epoch 15/20
103/103 _____ 0s 626ms/step - accuracy: 0.7027 - auc:
0.9933 - loss: 0.0787 - recall: 0.9285
Epoch 15: val_loss improved from 0.07144 to 0.07137, saving model
to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 15: finished saving model to /content/drive/MyDrive/X-Ray ML
Project/best_multilabel_model.keras
103/103 _____ 69s 674ms/step - accuracy: 0.7048 - auc:
0.9937 - loss: 0.0781 - recall: 0.9280 - val_accuracy: 0.7598 -
val_auc: 0.9941 - val_loss: 0.0714 - val_recall: 0.9210 -
learning_rate: 2.0000e-04

Epoch 16/20
103/103 _____ 0s 627ms/step - accuracy: 0.7080 - auc:
0.9945 - loss: 0.0744 - recall: 0.9271
Epoch 16: val_loss improved from 0.07137 to 0.07117, saving model
to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 16: finished saving model to /content/drive/MyDrive/X-Ray ML
Project/best_multilabel_model.keras
103/103 _____ 70s 673ms/step - accuracy: 0.7247 - auc:
0.9940 - loss: 0.0783 - recall: 0.9278 - val_accuracy: 0.7500 -
val_auc: 0.9944 - val_loss: 0.0712 - val_recall: 0.9230 -
learning_rate: 2.0000e-04

Epoch 17/20
103/103 _____ 0s 623ms/step - accuracy: 0.6921 - auc:

```

0.9929 - loss: 0.0822 - recall: 0.9266
Epoch 17: val_loss did not improve from 0.07117
103/103 _____ 67s 650ms/step - accuracy: 0.6984 - auc:
0.9936 - loss: 0.0774 - recall: 0.9315 - val_accuracy: 0.7451 -
val_auc: 0.9941 - val_loss: 0.0717 - val_recall: 0.9210 -
learning_rate: 2.0000e-04
Epoch 18/20
103/103 _____ 0s 635ms/step - accuracy: 0.7218 - auc:
0.9948 - loss: 0.0725 - recall: 0.9344
Epoch 18: val_loss improved from 0.07117 to 0.07061, saving model
to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 18: finished saving model to /content/drive/MyDrive/X-Ray ML
Project/best_multilabel_model.keras
103/103 _____ 70s 675ms/step - accuracy: 0.7128 - auc:
0.9946 - loss: 0.0737 - recall: 0.9291 - val_accuracy: 0.7451 -
val_auc: 0.9942 - val_loss: 0.0706 - val_recall: 0.9260 -
learning_rate: 2.0000e-04
Epoch 19/20
103/103 _____ 0s 638ms/step - accuracy: 0.7123 - auc:
0.9947 - loss: 0.0744 - recall: 0.9302
Epoch 19: val_loss did not improve from 0.07061
103/103 _____ 71s 689ms/step - accuracy: 0.7103 - auc:
0.9943 - loss: 0.0763 - recall: 0.9301 - val_accuracy: 0.7402 -
val_auc: 0.9945 - val_loss: 0.0707 - val_recall: 0.9250 -
learning_rate: 2.0000e-04
Epoch 20/20
103/103 _____ 0s 653ms/step - accuracy: 0.6989 - auc:
0.9940 - loss: 0.0747 - recall: 0.9311
Epoch 20: val_loss improved from 0.07061 to 0.07022, saving model
to /content/drive/MyDrive/X-Ray ML Project/best_multilabel_model.keras

Epoch 20: finished saving model to /content/drive/MyDrive/X-Ray ML
Project/best_multilabel_model.keras
103/103 _____ 72s 693ms/step - accuracy: 0.7051 - auc:
0.9942 - loss: 0.0748 - recall: 0.9303 - val_accuracy: 0.7279 -
val_auc: 0.9942 - val_loss: 0.0702 - val_recall: 0.9220 -
learning_rate: 2.0000e-04

```

Step 5 (A) : Model Loading

```

checkpoint_path = '/content/drive/MyDrive/X-Ray ML
Project/best_multilabel_model.keras'
log_path = '/content/drive/MyDrive/X-Ray ML Project/training_log.csv'

# 1. Load the Model Weights
if os.path.exists(checkpoint_path):
    model = load_model(checkpoint_path)

```

```

    print("Best model weights loaded from Drive.")
else:
    print("No saved model found. Please run Step 5 training first.")

# 2. Load the Training Logs (for Step 5b plotting)
if os.path.exists(log_path):
    history_df = pd.read_csv(log_path)
    print("Training logs loaded. You can now run Step 5(b) to see graphs.")
else:
    history_df = None
    print("No log file found. History plots will only work if you train in this session.")

Best model weights loaded from Drive.
Training logs loaded. You can now run Step 5(b) to see graphs.

```

Step 5 (B) : Training History (Learning Curve)

```

import matplotlib.pyplot as plt

def plot_learning_curves(data_source, is_df=False):
    """
    data_source: can be the 'history' object or 'history_df'
    dataframe.
    is_df: True if loading from CSV, False if plotting directly after
    training.
    """
    # Extract data based on source
    if is_df:
        loss = data_source['loss']
        val_loss = data_source['val_loss']
        acc = data_source['accuracy']
        val_acc = data_source['val_accuracy']
    else:
        loss = data_source.history['loss']
        val_loss = data_source.history['val_loss']
        acc = data_source.history['accuracy']
        val_acc = data_source.history['val_accuracy']

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

    # 1. Loss Plot
    ax1.plot(loss, label='Training Loss', color='#1f77b4',
linewidth=2)
    ax1.plot(val_loss, label='Validation Loss', color='#ff7f0e',
linestyle='--', linewidth=2)
    ax1.set_title('Model Loss (Cross-Entropy)')
    ax1.set_xlabel('Epoch')
    ax1.set_ylabel('Loss Value')

```

```

ax1.legend(); ax1.grid(True, alpha=0.3)

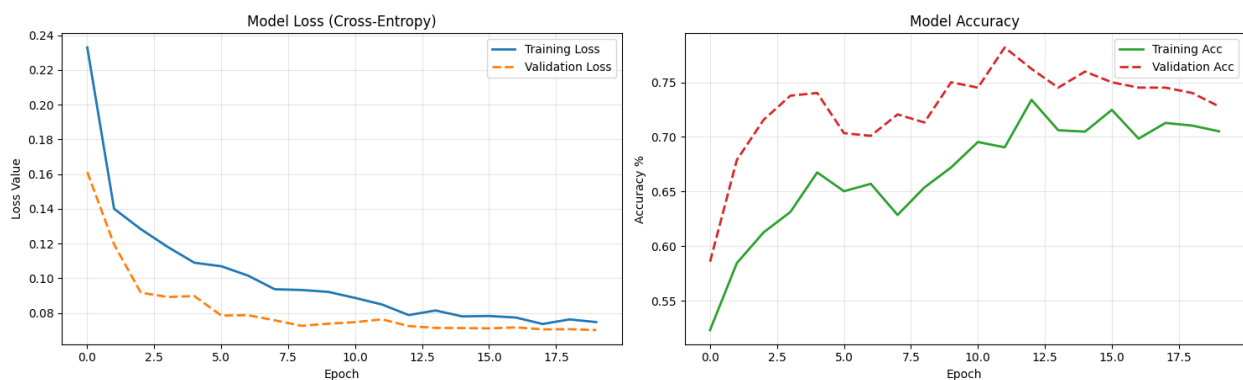
# 2. Accuracy Plot
ax2.plot(acc, label='Training Acc', color='#2ca02c', linewidth=2)
ax2.plot(val_acc, label='Validation Acc', color='#d62728',
linestyle='--', linewidth=2)
ax2.set_title('Model Accuracy')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Accuracy %')
ax2.legend(); ax2.grid(True, alpha=0.3)

plt.suptitle("Deep Learning Convergence Diagnostics", fontsize=16,
fontweight='bold', y=1.05)
plt.tight_layout()
plt.savefig('learning_curves.png', dpi=300) # Automatically saves
for Overleaf!
plt.show()

# --- EXECUTION ---
if 'history' in locals():
    # If we just trained, use the live history object
    plot_learning_curves(history, is_df=False)
elif 'history_df' in locals() and history_df is not None:
    # If we loaded from Drive, use the dataframe
    plot_learning_curves(history_df, is_df=True)
else:
    print(" Error: No training data found to plot.")

```

Deep Learning Convergence Diagnostics



Step 5(c) : HyperTuning with ensemble learning methods : Random Forest

```

print("--- Phase 1: Building the Hybrid ML Ensemble (CORRECTED) ---")

# 1. Create a Feature Extractor
feature_extractor = tf.keras.Model(inputs=model.input,
outputs=model.layers[-2].output)
print("Aligning data pipelines...")
extract_gen = test_datagen.flow_from_dataframe(
    train_df,
    x_col='image_path',
    y_col=target_columns,
    target_size=(224, 224),
    batch_size=32,
    class_mode='raw',
    shuffle=False
)

# 3. Extract Features safely
print("Extracting features from Train Set (Unshuffled)...")
train_features = feature_extractor.predict(extract_gen, verbose=1)
train_labels = train_df[target_columns].values

print("\nExtracting features from Test Set...")
test_gen.reset()
test_features = feature_extractor.predict(test_gen, verbose=1)
test_labels = test_df[target_columns].values

# 4. Train the Random Forest Ensemble
print("\nTraining Random Forest Meta-Classifier (Ensemble
Learning)...")
rf_base = RandomForestClassifier(n_estimators=100, max_depth=12,
n_jobs=-1, random_state=42)
ml_ensemble = MultiOutputClassifier(rf_base)

ml_ensemble.fit(train_features, train_labels)

print("Hybrid Ensemble successfully retrained with aligned data!")

--- Phase 1: Building the Hybrid ML Ensemble (CORRECTED) ---
Aligning data pipelines...
Found 3266 validated image filenames.
Extracting features from Train Set (Unshuffled)...
103/103 _____ 56s 426ms/step

Extracting features from Test Set...
13/13 _____ 17s 1s/step

Training Random Forest Meta-Classifier (Ensemble Learning)...
Hybrid Ensemble successfully retrained with aligned data!

```

Step 6: Evaluation & Performance Metrics

Evaluates the best saved model on the unseen Test Set and generates the Confusion Matrix.

```
# --- 6.1 Predict on Test Set ---
print("Evaluating on Test Set (Multi-Label)...")
test_gen.reset()

# Get predictions for all 11 columns
pred_probs = model.predict(test_gen, verbose=1)

# Convert probabilities to binary (0 or 1) based on 0.5 threshold
preds_binary = (pred_probs > 0.5).astype(int)

# Get the actual ground truth labels from the dataframe
# (Safe because shuffle=False in test_gen)
actuals = test_df[target_columns].values

# --- 6.2 Per-Class Classification Report ---
print("\n Multi-Label Classification Report:")
# This prints Precision, Recall, and F1-Score for EVERY category
# (Hand, Hip, Fracture, etc.)
print(classification_report(actuals, preds_binary,
target_names=target_columns))

# --- 6.3 Focused Confusion Matrix (Fracture Only) ---
# Although we predict 11 things, the "Fracture" check is the most
important.
# We extract the column corresponding to 'fractured' for the heatmap.

fracture_idx = target_columns.index('fractured')
actual_fractures = actuals[:, fracture_idx]
pred_fractures = preds_binary[:, fracture_idx]

Evaluating on Test Set (Multi-Label)...
13/13 ————— 27s 1s/step
```

Multi-Label Classification Report:

	precision	recall	f1-score	support
hand	0.99	0.97	0.98	154
leg	0.97	0.99	0.98	217
hip	0.95	1.00	0.98	40
shoulder	0.95	1.00	0.97	36
mixed	0.80	0.92	0.85	38
hardware	0.88	0.78	0.82	9
multiscan	0.97	0.94	0.96	35
fractured	0.77	0.38	0.50	72
frontal	0.96	0.95	0.95	252
lateral	0.99	0.94	0.96	151

oblique	0.74	0.81	0.77	36
micro avg	0.95	0.92	0.93	1040
macro avg	0.90	0.88	0.88	1040
weighted avg	0.94	0.92	0.92	1040
samples avg	0.96	0.93	0.94	1040

Step 7: Explainable AI (Grad-CAM Visualization)

Generates Heatmaps to visually interpret the model's focus areas.

```
def make_progressive_heatmap(img_array, model, layer_name, label_idx):
    # 1. Map input to the specific layer AND final output
    grad_model = tf.keras.models.Model(
        [model.inputs], [model.get_layer(layer_name).output,
        model.output]
    )

    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(img_array)

        # FIX: The source of your previous TypeError. If predictions
is a list, grab the tensor.
        if isinstance(predictions, list):
            predictions = predictions[0]

        loss = predictions[:, label_idx]

        # 2. Extract gradients for the specific class
        output = conv_outputs[0]
        grads = tape.gradient(loss, conv_outputs)[0]

        # 3. Weight channels by gradient importance
        gate_f = tf.reduce_mean(grads, axis=(0, 1))
        heatmap = output @ gate_f[..., tf.newaxis]
        heatmap = tf.squeeze(heatmap)

        # 4. Normalize
        heatmap = tf.maximum(heatmap, 0) / tf.math.reduce_max(heatmap)
        return heatmap.numpy()

def plot_evolution_of_attention(sample_idx):
    # Prepare the image
    row = test_df.iloc[sample_idx]
    img_path = row['image_path']
    img = tf.keras.preprocessing.image.load_img(img_path,
        target_size=(224, 224))
```

```

img_array = tf.keras.preprocessing.image.img_to_array(img)
model_input = np.expand_dims(img_array, axis=0) / 255.0

# We focus on 'fractured' index for the progression story
fracture_idx = target_columns.index('fractured')

# FIX: Updated layer names to match your specific TensorFlow
version
layers = ['conv1_relu', 'pool2_conv', 'relu']
titles = ['Stage 1: Geometric Outlines', 'Stage 2: Anatomical
ROI', 'Stage 3: Pathological Focus']

plt.figure(figsize=(20, 6))

# Plot Original
plt.subplot(1, 4, 1)
plt.imshow(img)
plt.title(f"Input Radiograph\n({row['image_id']})",
fontweight='bold')
plt.axis('off')

# Plot the Evolution
for i, layer_name in enumerate(layers):
    heatmap = make_progressive_heatmap(model_input, model,
layer_name, fracture_idx)

    # Resize and Overlay
    heatmap_res = cv2.resize(heatmap, (224, 224))
    heatmap_color = plt.cm.jet(heatmap_res)[: , : , :3]
    superimposed = (heatmap_color * 0.4) + (img_array / 255.0)
    superimposed = np.clip(superimposed, 0, 1)

    plt.subplot(1, 4, i + 2)
    plt.imshow(superimposed)
    plt.title(titles[i], color='darkred', fontweight='bold')
    plt.axis('off')

plt.suptitle("Hierarchical Attention Progression for Multi-Label
Classification", fontsize=16, y=1.05)
plt.tight_layout()
plt.show()

# --- EXECUTION ---
# Pick a fracture case from the test set
fracture_cases = np.where(test_df['fractured'] == 1)[0]
if len(fracture_cases) > 0:
    plot_evolution_of_attention(fracture_cases[8])
    plot_evolution_of_attention(fracture_cases[9])
    plot_evolution_of_attention(fracture_cases[10])
    plot_evolution_of_attention(fracture_cases[11])

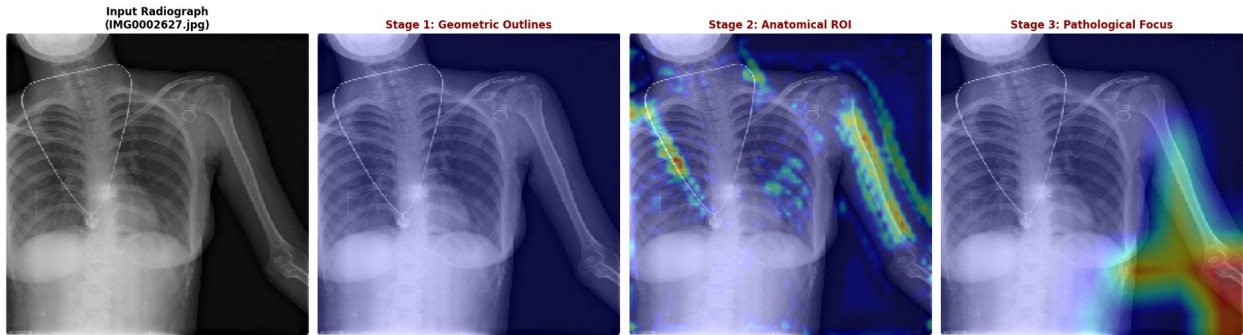
```

```

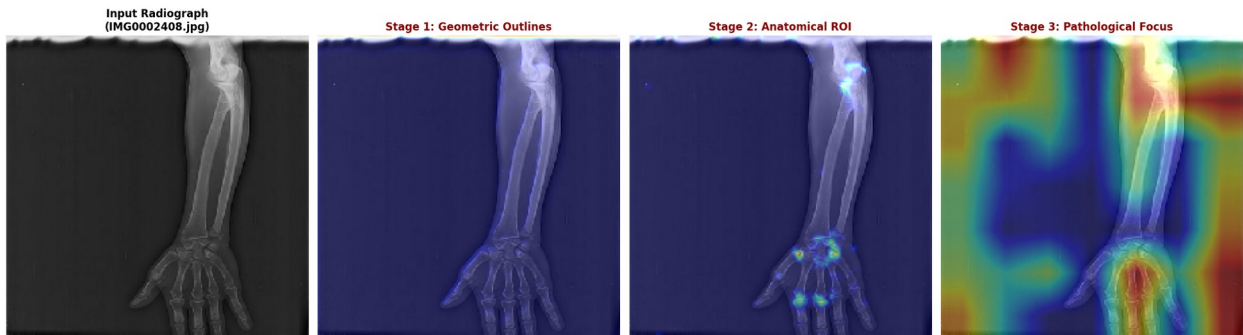
plot_evolution_of_attention(fracture_cases[12])
plot_evolution_of_attention(fracture_cases[13])
plot_evolution_of_attention(fracture_cases[14])
plot_evolution_of_attention(fracture_cases[15])
plot_evolution_of_attention(fracture_cases[16])

```

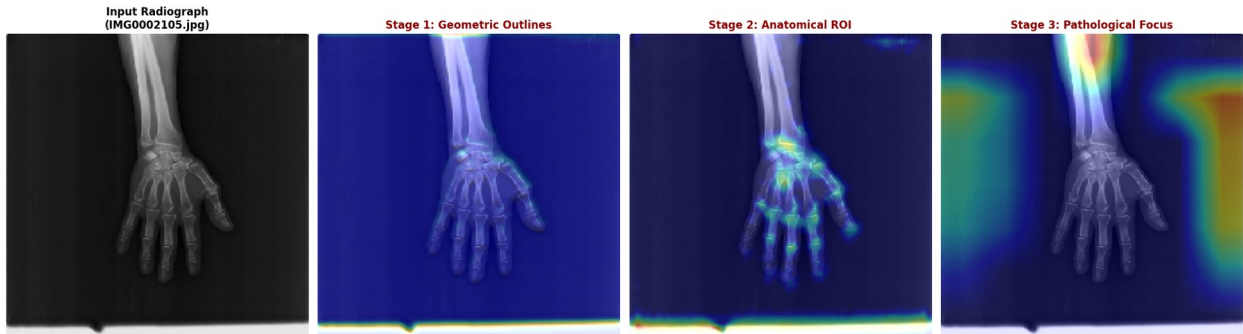
Hierarchical Attention Progression for Multi-Label Classification



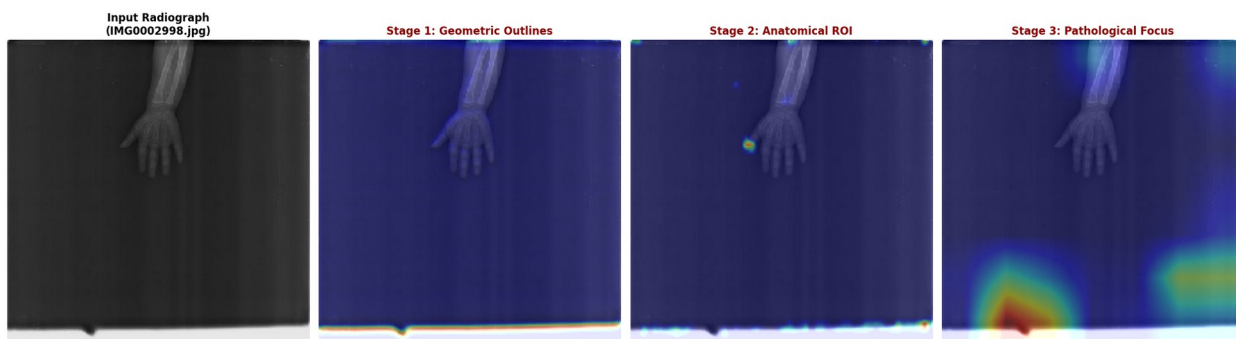
Hierarchical Attention Progression for Multi-Label Classification



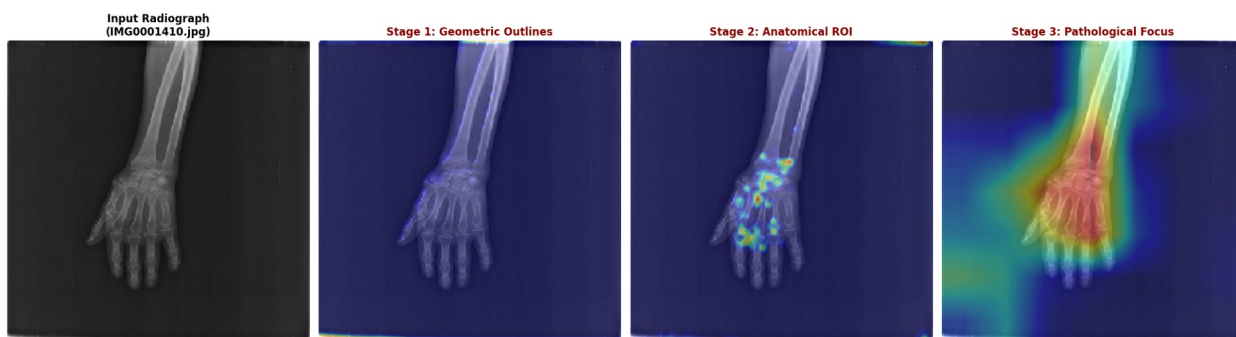
Hierarchical Attention Progression for Multi-Label Classification



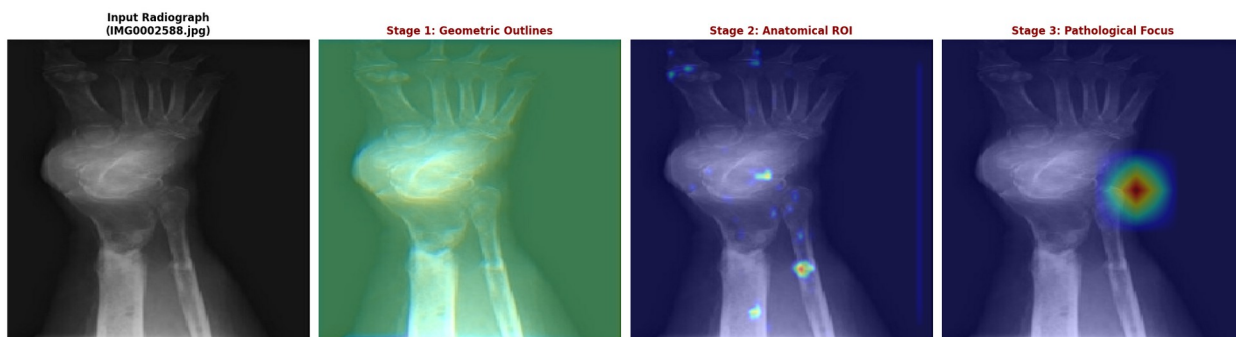
Hierarchical Attention Progression for Multi-Label Classification



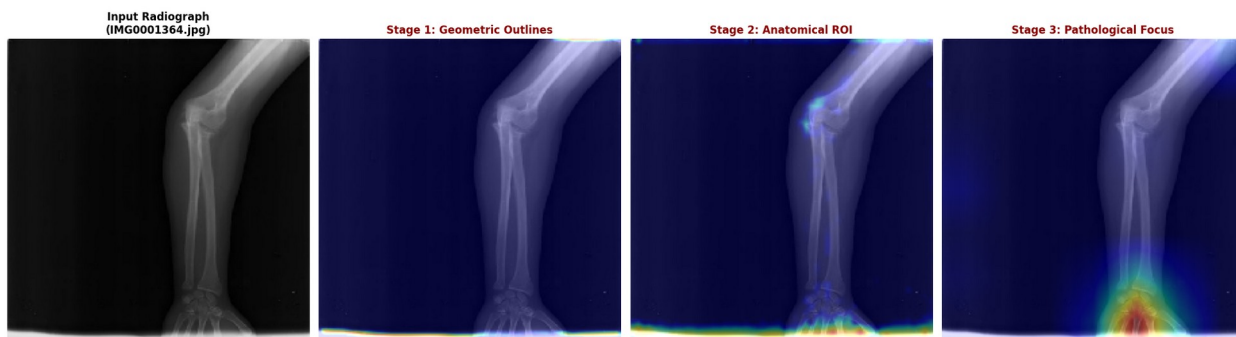
Hierarchical Attention Progression for Multi-Label Classification

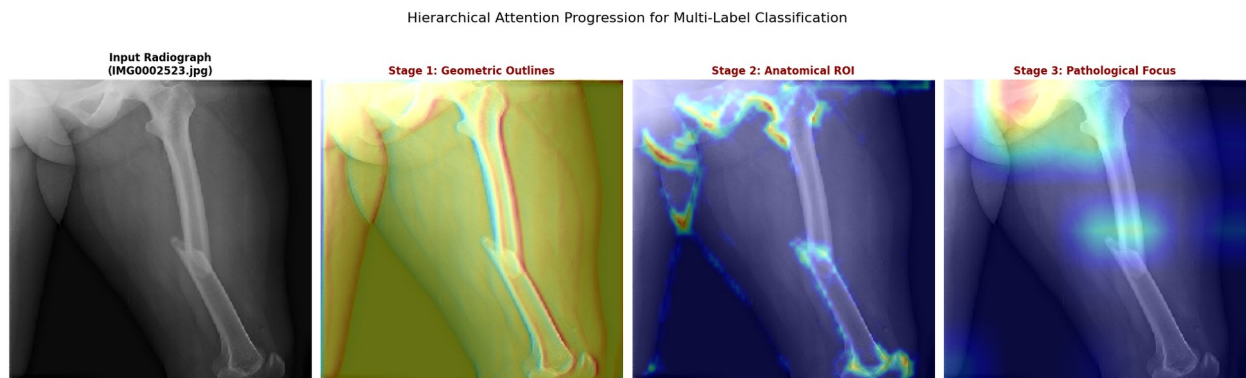
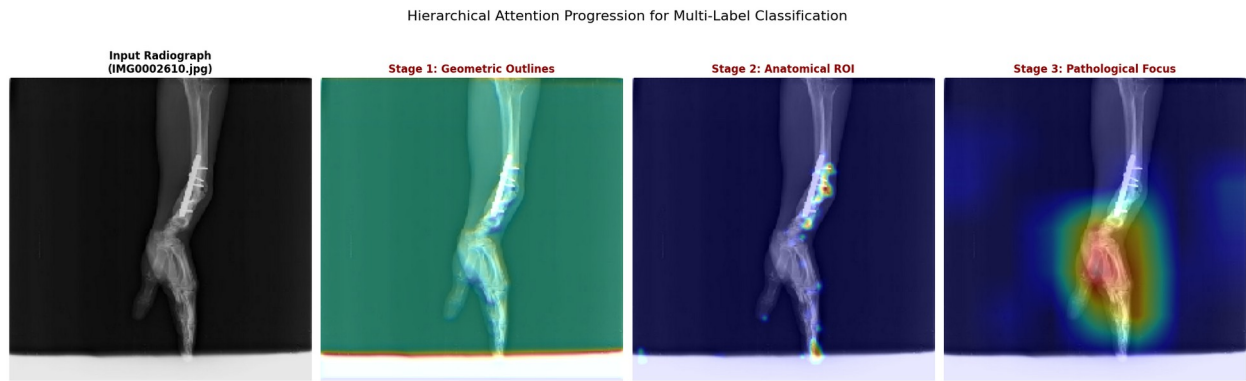


Hierarchical Attention Progression for Multi-Label Classification



Hierarchical Attention Progression for Multi-Label Classification





Step 7.b: Multi-Label Performance Dashboard

```
def plot_hybrid_performance_dashboard(actuals, preds_binary,
target_columns):
    print("Generating Hybrid Ensemble Performance Dashboard...")

    # 1. Calculate metrics for each of the 11 classes
    metrics = precision_recall_fscore_support(actuals, preds_binary,
average=None, labels=range(len(target_columns)))

    # 2. Build the DataFrame
    performance_df = pd.DataFrame({
        'Category': [c.capitalize() for c in target_columns],
        'Precision': metrics[0],
        'Recall': metrics[1],
        'F1-Score': metrics[2]
    })

    # Sort by F1-Score for a cleaner visual cascade
    performance_df = performance_df.sort_values('F1-Score',
ascending=False).set_index('Category')

    # 3. Plotting the grouped bar chart with an Aesthetic Palette
    # Palette: Sage Green (Precision), Muted Coral (Recall), Deep
    Slate (F1-Score)
```

```

aesthetic_colors = ['#7CA18B', '#E58A7B', '#34495E']

# We also remove the border line around the bars for a cleaner, modern look (edgecolor='none')
ax = performance_df.plot(kind='bar', figsize=(15, 6),
color=aesthetic_colors, width=0.8, edgecolor='none')

plt.title("Hybrid Ensemble: Multi-Label Clinical Performance Metrics", fontsize=16, fontweight='bold', color='#2C3E50')
plt.ylabel("Score (0.0 to 1.0)", fontsize=12, color='#2C3E50')
plt.xlabel("Diagnostic Classifications", fontsize=12, color='#2C3E50')

# Soften the axis text colors
plt.xticks(rotation=45, ha='right', fontsize=11, color='#4A4A4A')
plt.yticks(fontsize=11, color='#4A4A4A')
plt.ylim(0, 1.15)

# Clean, minimalistic legend without a harsh border
plt.legend(loc='lower center', bbox_to_anchor=(0.5, -0.3), ncol=3,
fontsize=12, frameon=False)

# Use a very subtle, dotted grid line instead of heavy dashes
plt.grid(axis='y', linestyle=':', alpha=0.6, color='#BDC3C7')

# Remove the top and right borders (spines) of the graph for a sleek "data viz" look
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_color('#BDC3C7')
ax.spines['bottom'].set_color('#BDC3C7')

# 4. Add the exact numbers on top of each bar (matching the bar color)
for i, p in enumerate(ax.patches):
    height = p.get_height()
    if height > 0:
        # Match the text color to the bar color for a cohesive aesthetic
        text_color = aesthetic_colors[i // len(target_columns)]

        ax.annotate(f"{height:.2f}",
                    (p.get_x() + p.get_width() / 2., height),
                    ha='center', va='bottom',
                    xytext=(0, 4),
                    textcoords='offset points',
                    fontsize=9, rotation=90, fontweight='bold',
                    color=text_color)

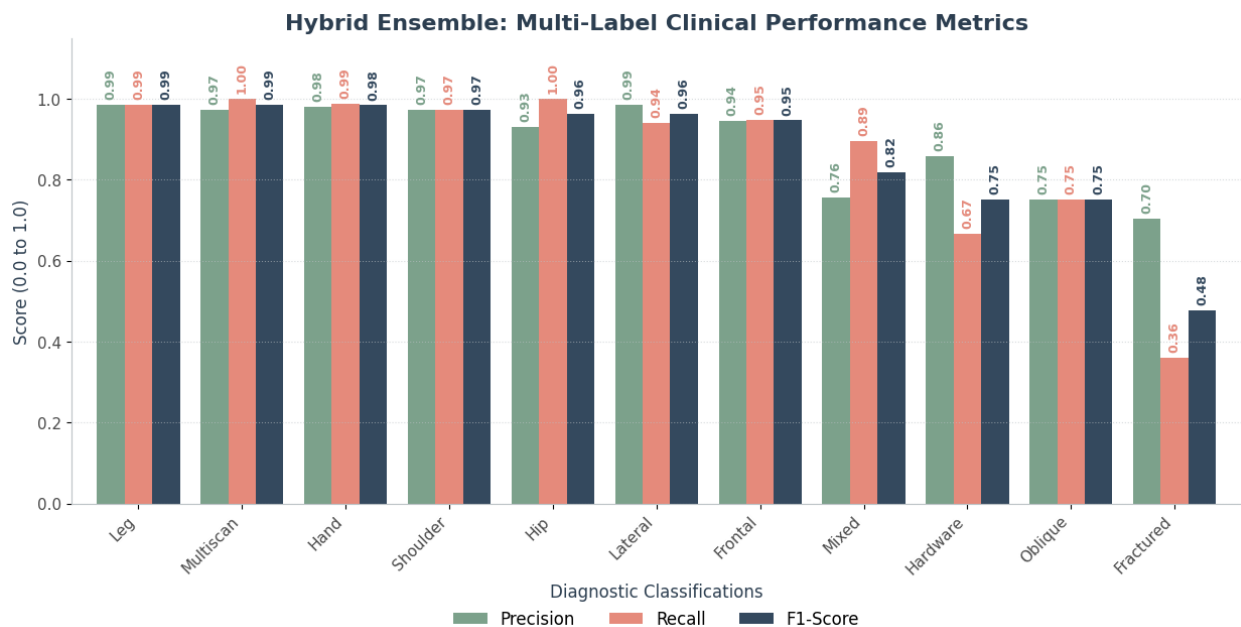
# 5. Automatically save for your research paper!

```

```
plt.savefig('hybrid_performance_dashboard.png', dpi=300,
bbox_inches='tight')
plt.show()
print("Dashboard saved as 'hybrid_performance_dashboard.png'")

# --- EXECUTION ---
# This uses the 'actuals' and 'preds_binary' generated in your
previous evaluation cell
plot_hybrid_performance_dashboard(actuals, preds_binary,
target_columns)
```

Generating Hybrid Ensemble Performance Dashboard...



Dashboard saved as 'hybrid_performance_dashboard.png'

Step 8: Live Testing Interface (User Upload)

This section allows manual uploading of external X-rays to test the model in a real-world scenario. It generates a diagnostic report and visual evidence (Heatmap) for each uploaded file.

```
# --- 1. Helper Function: Robust Heatmap Generator ---
def make_gradcam_heatmap(img_array, model, last_conv_layer_name,
pred_index=None):
    grad_model = tf.keras.models.Model(
        [model.inputs],
        [model.get_layer(last_conv_layer_name).output, model.output]
    )
```

```

with tf.GradientTape() as tape:
    last_conv_layer_output, preds = grad_model(img_array)

    # FIX: Handle list outputs to prevent TypeError
    if isinstance(preds, list):
        preds = preds[0]

    # For multi-label, focus on the specific class index
    if pred_index is None:
        pred_index = tf.argmax(preds[0])
    class_channel = preds[:, pred_index]

    grads = tape.gradient(class_channel, last_conv_layer_output)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    last_conv_layer_output = last_conv_layer_output[0]
    heatmap = last_conv_layer_output @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
    heatmap = tf.maximum(heatmap, 0) / tf.math.reduce_max(heatmap)
    return heatmap.numpy()

# --- 2. Main Diagnostic Function (HYBRID UPGRADE + VERDICT) ---
def generate_full_report(uploaded_filename):
    print(f"\nProcessing {uploaded_filename} through Hybrid Ensemble...")

    # A. Preprocess Image
    img = tf.keras.preprocessing.image.load_img(uploaded_filename,
target_size=(224, 224))
    img_array = tf.keras.preprocessing.image.img_to_array(img)

    # Medical CLAHE Logic
    img_processed = img_array.astype('uint8')
    gray = cv2.cvtColor(img_processed, cv2.COLOR_RGB2GRAY)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    enhanced = clahe.apply(gray)
    img_final = np.stack((enhanced,)*3, axis=-1).astype('float32') /
255.0
    model_input = np.expand_dims(img_final, axis=0)

    # B. HYBRID PREDICTION LOGIC
    # 1. Extract Deep Features using DenseNet
    features = feature_extractor.predict(model_input, verbose=0)

    # 2. Get probabilities from the Random Forest
    rf_probs = ml_ensemble.predict_proba(features)
    rf_predictions = [prob[0][1] if prob.shape[1] > 1 else 0 for prob
in rf_probs]

    # C. Build the Clinical Table & FINAL VERDICT

```

```

results = []
for i, label in enumerate(target_columns):
    score = rf_predictions[i]
    status = "YES" if score > 0.5 else "---"
    results.append({
        "Category": label.upper(),
        "RF Confidence": f"{score*100:.1f}%",
        "Status": status
    })

df_report = pd.DataFrame(results)

# >>> THE NEW FINAL VERDICT LOGIC <<<
fracture_index = target_columns.index('fractured')
fracture_score = rf_predictions[fracture_index]

print("\n" + "="*55)
if fracture_score > 0.5:
    print(f" FINAL VERDICT: FRACTURE DETECTED
({fracture_score*100:.1f}% Confidence)")
else:
    print(f" FINAL VERDICT: FRACTURE NOT DETECTED")
print("="*55 + "\n")

# D. Explainable AI (Grad-CAM using DenseNet Backbone)
last_conv_layer = [l.name for l in model.layers if 'conv' in
l.name or 'bn' in l.name][-1]
heatmap = make_gradcam_heatmap(model_input, model,
last_conv_layer, pred_index=fracture_index)

# Overlay
heatmap_resized = cv2.resize(heatmap, (224, 224))
heatmap_color = plt.cm.jet(heatmap_resized)[:, :, :3]
superimposed_img = (heatmap_color * 0.4) + img_final
superimposed_img = np.clip(superimposed_img, 0, 1)

# E. Display Dashboard
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1); plt.imshow(img_array.astype('uint8'));
plt.title(f"Original Input"); plt.axis('off')
plt.subplot(1, 2, 2); plt.imshow(superimposed_img); plt.title("AI
Attention (Fracture Focus)"); plt.axis('off')
plt.show()

print("\nCOMPREHENSIVE RADIOLOGY REPORT (HYBRID AI):")
# Stylized display

display(df_report.style.hide(axis="index").set_properties(**{'text-
align': 'left'}))
print("-" * 65)

```

```
# --- 3. EXECUTION ---
print("Upload an X-ray to generate a Full Diagnostic Report")
uploaded = files.upload()

for fn in uploaded.keys():
    generate_full_report(fn)

Upload an X-ray to generate a Full Diagnostic Report
<IPython.core.display.HTML object>
Saving IMG0001980.jpg to IMG0001980.jpg
Processing IMG0001980.jpg through Hybrid Ensemble...

=====
FINAL VERDICT: FRACTURE NOT DETECTED
=====

/usr/local/lib/python3.12/dist-packages/keras/src/models/
functional.py:241: UserWarning: The structure of `inputs` doesn't
match the expected structure.
Expected: [['input_layer_1']]
Received: inputs=Tensor(shape=(1, 224, 224, 3))
warnings.warn(msg)
```

Original Input



AI Attention (Fracture Focus)



COMPREHENSIVE RADIOLOGY REPORT (HYBRID AI):

<pandas.io.formats.style.Styler at 0x7aea78713050>

```

def plot_comprehensive_evaluation(actuals, pred_probs, target_names):
    """
    Plots Confusion Matrix and ROC Curve specifically for the
    'Fracture' label.
    actuals: ground truth array
    pred_probs: model.predict() probabilities
    """
    # Find the index for 'fractured'
    frac_idx = target_names.index('fractured')
    y_true = actuals[:, frac_idx]
    y_score = pred_probs[:, frac_idx]
    y_pred = (y_score > 0.5).astype(int)

    # --- 1. Confusion Matrix (Fracture) ---
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
    plt.title('Confusion Matrix: Fracture')
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')

    # --- 2. ROC Curve (Fracture) ---
    fpr, tpr, _ = roc_curve(y_true, y_score)
    roc_auc = auc(fpr, tpr)

    plt.subplot(1, 3, 2)
    plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC area = {roc_auc:.2f}')
    plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
    plt.xlim([0.0, 1.0]); plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate (1 - Specificity)')
    plt.ylabel('True Positive Rate (Sensitivity)')
    plt.title('ROC Curve: Fracture Detection')
    plt.legend(loc="lower right")

    # --- 3. Precision-Recall Curve (Fracture) ---
    precision, recall, _ = precision_recall_curve(y_true, y_score)

    plt.subplot(1, 3, 3)
    plt.plot(recall, precision, color='green', lw=2)
    plt.xlabel('Recall (Sensitivity)')
    plt.ylabel('Precision (Positive Predictive Value)')
    plt.title('PR Curve: Fracture Detection')

    plt.tight_layout()

```

```

plt.savefig('performance_metrics.png', dpi=300)
plt.show()

if 'actuals' in locals() and 'pred_probs' in locals():
    plot_comprehensive_evaluation(actuals, pred_probs, target_columns)
else:
    print("Error: You must run model.predict on the test set first to
    get 'actuals' and 'pred_probs'.")

```

